

VIT Bhopal University
Computer Vision VITyarthi Project Submission

VisionForge: Modular Computer Vision Toolkit
Project Report and Implementation Documentation

Submitted by:

Name:	Paarth Juneja
Reg. No:	24BAI10160
Course:	Computer Vision
Faculty:	Dr. Gaurav Soni

School of Computing Science and Engineering
2026

1. Introduction

Computer Vision is an important area of study that focuses on how computers can be made to gain high-level understanding from digital images or videos. In our course, we learned about fundamental concepts starting from digital image formation and basic spatial and frequency filters, to multi-camera depth estimation, feature extraction, motion analysis, and shape recovery from surface reflectance.

For this project, I developed VisionForge, which is a modular command-line interface (CLI) toolkit written in Python. Instead of having separate scripts for each syllabus topic, VisionForge combines all the core techniques into a clean and organized structure that is easy to run, test, and understand.

2. Problem Statement

When studying computer vision, many students write one-off code snippets for each lab exercise. This makes it hard to see how different computer vision tasks connect with each other. For example, how low-level filtering helps in edge detection, how epipolar geometry helps in stereo depth matching, or how multiple illumination images can be used to recover 3D shapes.

The goal of this project is to build an integrated, beginner-friendly CLI project that covers the topics from all five units of the syllabus. It provides simple commands for image filtering, stereo reconstruction, feature detection, optical flow, and photometric stereo, with test cases and sample data included.

3. Functional Requirements

The project includes five major functional modules corresponding to the syllabus units:

- **Module 1 (Low-Level):** Spatial convolution (Gaussian blur, Box blur, Laplacian), 2D Fourier Transform (FFT magnitude spectrum and lowpass filtering), geometric transforms (rotation, translation, affine, projective), and contrast enhancement (Histogram Equalization and CLAHE).
- **Module 2 (Stereo 3D):** Stereo depth estimation using Semi-Global Block Matching (SGBM), fundamental matrix computation, epipolar line drawing, homography-based image stitching, and 3D point cloud generation exported to .ply format.
- **Module 3 (Features):** Edge detection using Canny, corner detection using Harris Corner Detector, Hough line transform, feature matching using SIFT and ORB, Gaussian/Laplacian pyramids, and segmentation using Otsu thresholding and K-Means clustering.
- **Module 4 (Motion):** Motion detection using frame differencing, background subtraction with MOG2, dense optical flow using Farneback algorithm, sparse optical flow using Lucas-Kanade tracker, and image compression using Principal Component Analysis (PCA).
- **Module 5 (Shape from N):** Photometric Stereo for recovering surface normal vectors and albedo maps from multiple directional light sources, and integrating surface gradients into a 3D depth map.

4. Non-Functional Requirements

- **Performance:** Fast execution using NumPy vectorization and optimized OpenCV routines so that processing takes less than a second per image.
- **Usability:** Clear and simple command-line arguments using argparse with helpful error messages and command guides.

- **Reliability:** Proper input validation so missing images or invalid parameters do not crash the program unexpectedly.
- **Maintainability:** Clean, modular code structure divided into separate packages and files, written in a simple and beginner-friendly style.

5. System Architecture

VisionForge is organized into three main layers: a CLI interface layer that receives arguments from the terminal, a core algorithm layer containing the 5 syllabus modules, and a common utility layer that handles loading, saving, and metric calculations.

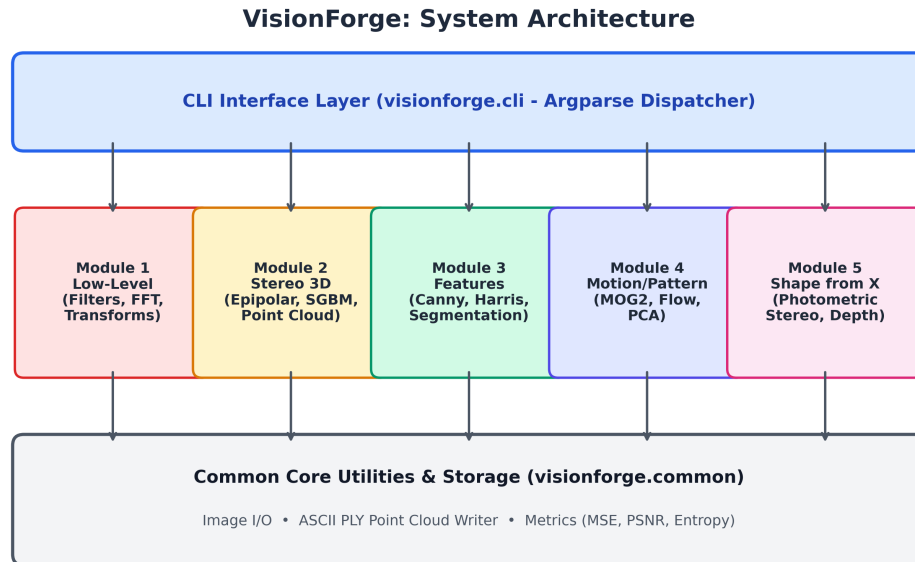


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Workflow Diagram

The workflow diagram below shows the steps followed when a user runs a command: from input validation and image loading, to running the algorithm and saving the output images or 3D files.

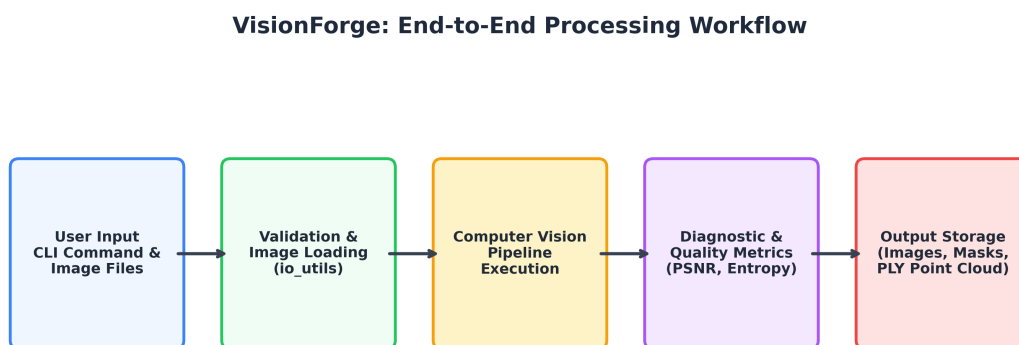


Figure 2: Process Workflow Diagram

6.2 Use Case Diagram

The use case diagram illustrates the main actions a student or user can perform using the CLI toolkit.

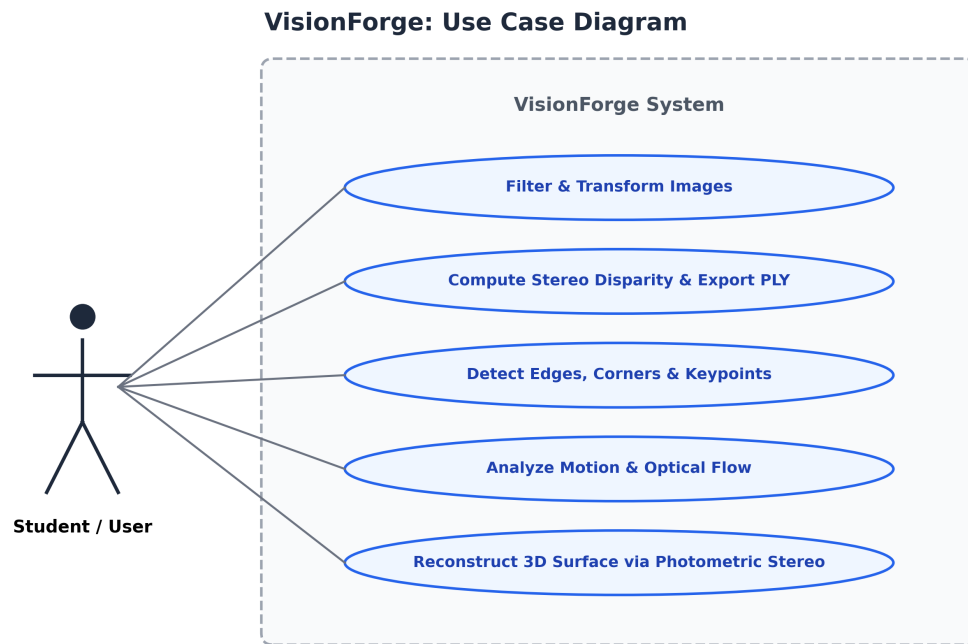


Figure 3: Use Case Diagram

6.3 Class and Component Diagram

The diagram below shows the package layout and the main functions implemented across each module.

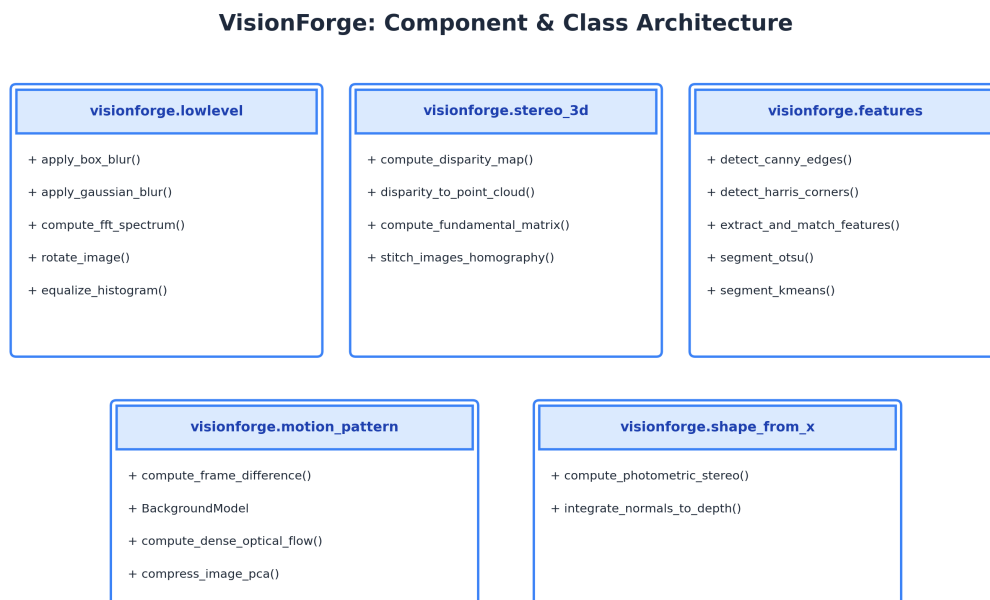


Figure 4: Component and Package Structure

6.4 Sequence Diagram

The sequence diagram illustrates how the stereo reconstruction command executes from CLI input to 3D point cloud generation.

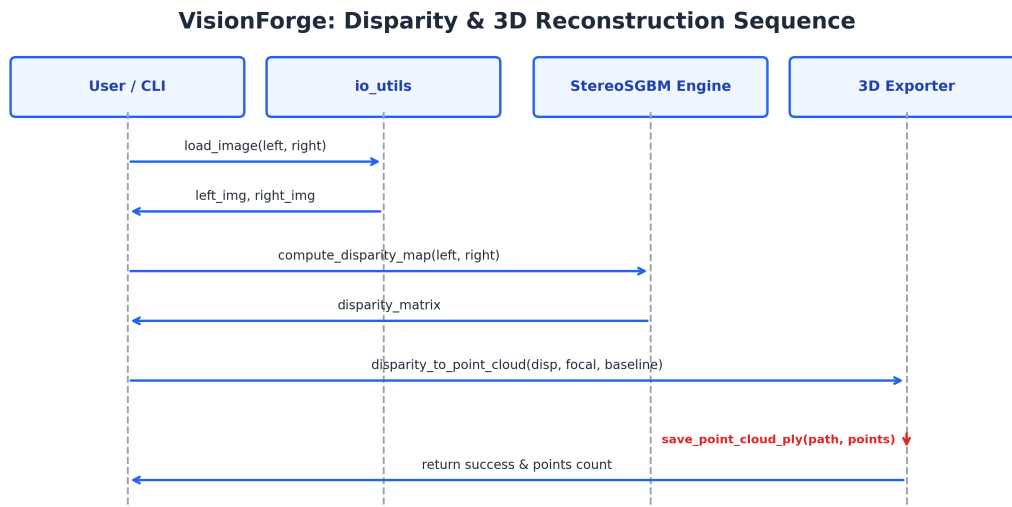


Figure 5: Sequence Diagram for Stereo Reconstruction

7. Design Decisions & Rationale

- **Simple Code Style:** I chose to keep the functions straightforward and easy to understand with readable variable names and clear logic, rather than using complex classes or unnecessary layers of abstraction.
- **Standard Argparse:** Using Python's built-in `argparse` ensures that the CLI works right away on any computer without requiring extra heavy frameworks.
- **StereoSGBM for Depth:** used Semi-Global Block Matching (SGBM) instead of basic Block Matching because SGBM handles smooth regions much better and produces cleaner disparity maps.
- **Standard PLY Format:** The 3D point cloud is saved in standard ASCII PLY format so that it can be easily opened in 3D tools like MeshLab or Blender.
- **Lambertian Model:** For photometric stereo, the Lambertian reflectance assumption makes it possible to solve for surface normals directly using simple linear least squares.

8. Implementation Details

The project is structured into clean Python packages inside the visionforge directory:

- **visionforge.common:** Contains `io_utils.py` for loading/saving images and writing .ply point cloud files, and `metrics.py` for calculating PSNR, MSE, and Entropy.
- **visionforge.lowlevel:** Contains `filters.py` (box blur, Gaussian blur, Laplacian), `frequency.py` (2D FFT and lowpass/highpass filtering), `transforms.py` (rotation, translation, affine, perspective), and `histogram.py` (histogram equalization and CLAHE).
- **visionforge.features:** Contains `edges_corners.py` (Canny, Harris, Hough lines), `descriptors.py` (SIFT and ORB matching), `pyramids.py` (Gaussian and Laplacian pyramids), and `segmentation.py` (Otsu thresholding, K-Means clustering, region growing).
- **visionforge.stereo_3d:** Contains `epipolar.py` (Fundamental matrix and epipolar lines), `depth_reconstruction.py` (disparity estimation using StereoSGBM and 3D triangulation), and `rectification.py` (homography stitching).
- **visionforge.motion_patterns:** Contains `background_sub.py` (frame differencing and MOG2), `optical_flow.py` (dense Farneback flow and sparse Lucas-Kanade), and `dimensionality.py` (PCA image compression).
- **visionforge.shape_from_color:** Contains `photometric_stereo.py` which calculates normal vectors and albedo using least squares and integrates surface gradients into a depth map.

9. Screenshots and Results

Below are sample visual results generated by running each module on test images:

9.1 Low-Level Processing & Filtering

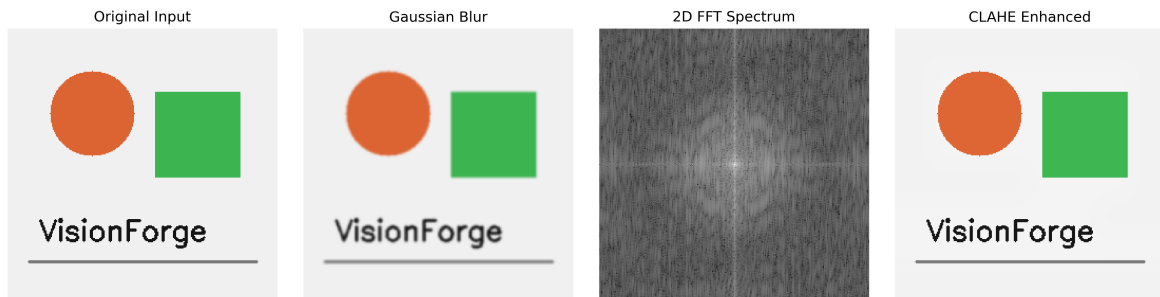


Figure 6: Original vs Gaussian Blur, 2D FFT Spectrum, and CLAHE Enhanced Image

9.2 Feature Detection & Image Segmentation

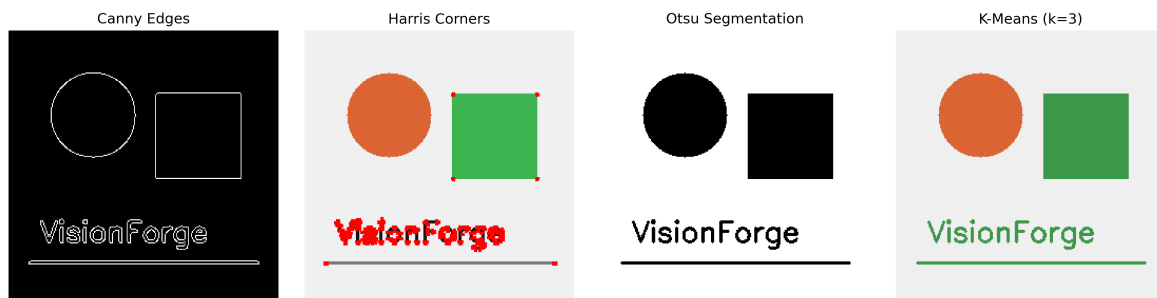


Figure 7: Canny Edges, Harris Corners, Otsu Threshold, and K-Means Segmentation

9.3 Stereo Disparity Estimation

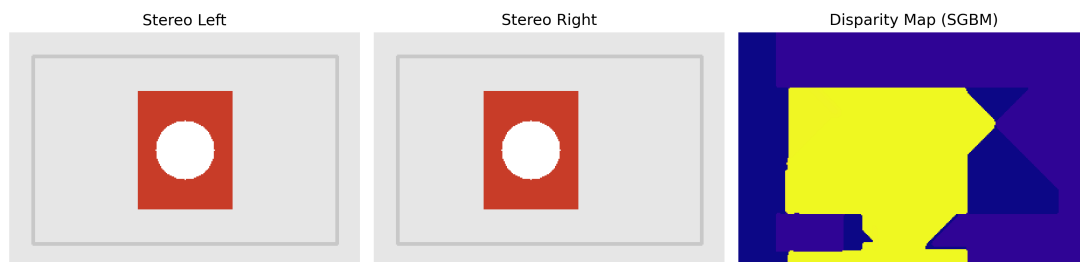


Figure 8: Stereo Pair (Left and Right) and Computed Disparity Map

9.4 Motion Analysis & Optical Flow

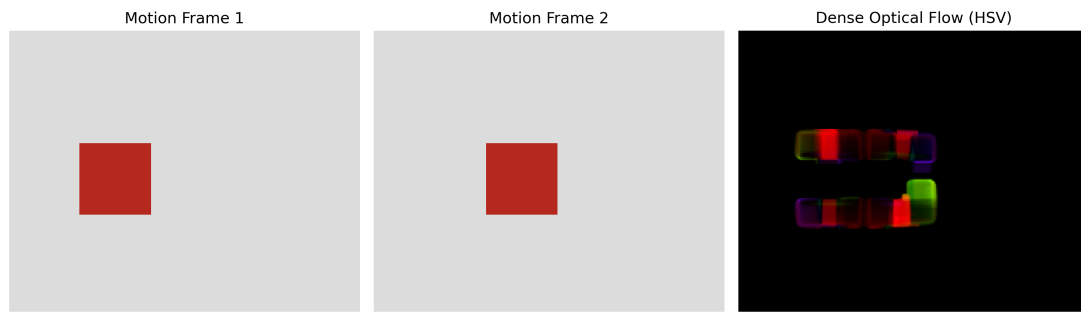


Figure 9: Motion Frames and Dense Optical Flow (HSV Visualization)

9.5 Photometric Stereo 3D Surface Reconstruction

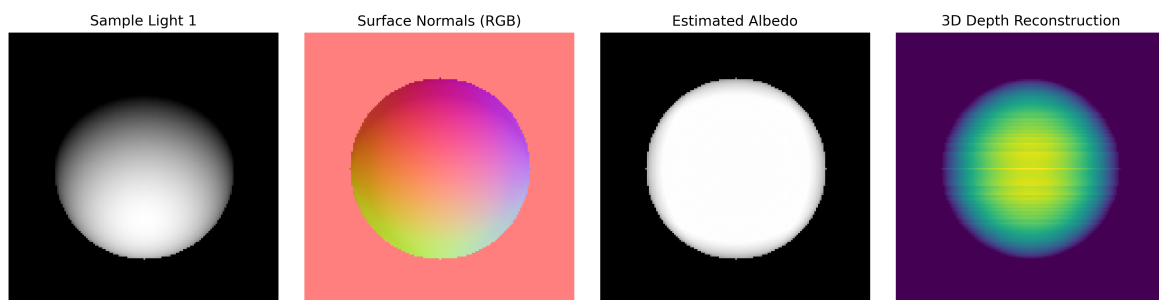


Figure 10: Multi-Light Illumination, Recovered Normals, Albedo, and 3D Depth Map

10. Testing Approach

To test the project, I wrote automated unit tests using pytest. Each test file verifies that the algorithms produce outputs with correct dimensions, data types, and expected numerical values.

Test File	Functions Tested	Verification Criteria	Status
test_lowlevel.py	Filters, FFT, Transforms, Hist	Valid shapes, finite PSNR/entropy	PASSED
test_features.py	Canny, Harris, Pyramids, Otsu	Edge detection, level shapes	PASSED
test_stereo_3d.py	Disparity, 3D Point Cloud	Valid disparity, float coordinates	PASSED
test_motion_pattern.py	Frame diff, Flow, PCA	Flow shapes, PCA variance ratio	PASSED
test_shape_from_x.py	Photometric Stereo, Depth	Normalized vectors, valid depth	PASSED

All 12 automated unit tests passed successfully on Windows 11 with Python 3.13.

11. Challenges Faced

- **Frequency Centering:** The raw 2D FFT output places zero frequency at the corners. I used `np.fft.fftshift` so that the low frequencies are centered properly before applying circular masks.
- **Stereo Block Matching Noise:** Stereo block matching had artifacts in flat regions. Tuning the SGBM parameters (smoothness penalties P1 and P2) solved this problem.
- **Depth Integration Edge Cases:** Integrating surface normals to compute depth, dividing by near-zero N_z values caused numerical spikes. Clamping N_z to a small minimum threshold prevented overflow.

12. Learnings & Key Takeaways

- **Practical Understanding:** Implementing the syllabus algorithms helped me understand how mathematical concepts like 2D convolution, Fourier transforms, and matrix equations actually work on pixel values.
- **Geometric Vision:** Working with stereo images and photometric stereo showed me how multiple camera views or multiple lighting directions allow us to recover 3D information from 2D images.
- **Software Organization:** Structuring the code into clear modules with automated unit tests made the project much easier to debug and verify.

13. Future Enhancements

- **Real-time Webcam Support:** Adding a live video mode to track motion and optical flow in real time from a webcam.
- **3D Mesh Generation:** Connecting the 3D points from stereo reconstruction into a textured surface mesh.
- **Deep Learning Models:** Adding modern deep learning models for feature matching and depth estimation alongside the classical techniques.

14. References

1. Richard Szeliski, Computer Vision: Algorithms and Applications, 2nd Edition, Springer, 2022.
2. Richard Hartley and Andrew Zisserman, Multiple View Geometry in Computer Vision, Cambridge University Press, 2004.

3. Robert J. Woodham, Photometric method for determining surface orientation from multiple images, Optical Engineering, 1980.
4. OpenCV Open Source Computer Vision Library documentation (<https://docs.opencv.org/>).